Workshop 3: Regression models

TECH6

Richard Foltyn
NHH Norwegian School of Economics

October 29, 2026

See GitHub repository for notebooks and data:

https://github.com/richardfoltyn/TECH6-H26

Contents

1 Exercise 1: Predicting house prices with linear models **1**

Exercise 1: Predicting house prices with linear models

In this exercise, we return to the Ames housing data from Lectures 1 and 2. We examine whether regularization can improve predictions of (log) sale prices over the OLS benchmark.

The following cell defines the global variables used throughout the notebook. You should change the number of jobs to run in parallel (`NUM_JOBS`) to adapt to your system.

[1]:
```
# Random state used for sample splitting (train/test, cross-validation) and other RNGs
RANDOM_STATE = 1234

# Scoring metric for cross-validation
SCORING = 'neg_root_mean_squared_error'

# Number of jobs to run in parallel (ADAPT TO YOUR SYSTEM!)
NUM_JOBS = 4
```

Data description

The following cell loads the Ames housing data from the file `data/ames_houses.csv`. The table below summarizes the variables used in this exercise.

[2]:
```
import pandas as pd

file = '../../data/ames_houses.csv'
df_ames = pd.read_csv(file)

print(f'Number of observations in the common sample: {len(df_ames):,d}')
```

```
Number of observations in the common sample: 2,930
```

The data contain the following house characteristics and sale information:

Variables	Description
`SalePrice`	Sale price, in thousands of USD
`LivingArea`, `LotArea`	Above-ground living area and lot area, in m²
`BasementArea`, `FinishedBasementArea`	Total and finished basement area, in m²
`YearBuilt`, `YearRemodeled`	Construction year and most recent remodel year
`OverallQuality`, `OverallCondition`	Ratings from 1 to 10
`ExteriorQuality`	Exterior material quality, recorded as text
`Bathrooms`, `Bedrooms`, `Fireplaces`	Numbers of rooms and fireplaces
`BuildingType`	Type of dwelling
`CentralAir`, `HasGarage`	Air conditioning and garage indicators
`YearSold`, `MonthSold`	Year and month of sale

Finished basement area is the part of total basement area finished as living space. Basement areas can be zero, so these variables remain in levels.

Part 1 — Prepare the data

As a first step, you need to prepare the data for modeling. The following steps are required:

1. Remove rows with missing values.
2. Remove houses with lot areas above 10,000 m² and living areas above 350 m².
3. Create `logSalePrice`, `logLivingArea`, and `logLotArea` using natural logs.
4. Split the full DataFrame into `df_train` and `df_test`, reserving 50% of the observations for testing with `random_state=1234`.
5. Define `y_train` and `y_test` using `logSalePrice` as the target.

Part 2 — Explore the training data

1. Use the provided `plot_histograms()` to compare the distributions of sale price, living area, and lot area in levels and logs. What changes after taking logs?
2. Use the provided `plot_scatterplots()` to plot log sale price against the numeric candidate predictors. Discuss the relationships, unusual observations, and possible predictive information.
3. Report each predictor's correlation with log price. Which associations are strongest, and how do they compare with the scatter plots?

Base this exploratory analysis on the training data only!

[7]:
```python
import matplotlib.pyplot as plt


def plot_histograms(df_data, title=None):
    """
    Plot a histogram for every column in a DataFrame.

    Parameters
    ----------
    df_data : DataFrame
        Numeric columns to plot
    title : str or None
        Optional title for the entire figure
    """
    ncols = df_data.shape[1]
    figsize = (3 * ncols, 2.8)
    fig, axes = plt.subplots(
```

```
        1, ncols, figsize=figsize, layout='constrained', squeeze=False
    )
    for i, col in enumerate(df_data.columns):
        ax = axes[0, i]
        ax.hist(df_data[col], bins=40, color='steelblue', ec='white', lw=0.4)
        ax.set_xlabel(col)
        ax.set_ylabel('Number of houses')

    if title is not None:
        fig.suptitle(title)
```

[8]:
```
def plot_scatterplots(y_var, df_data):
    """
    Plot one column against every other column in a DataFrame.

    Parameters
    ----------
    y_var : str
        Column name to use on the shared y-axis
    df_data : DataFrame
        Data containing the outcome and numeric predictors
    """
    columns = df_data.columns.drop(y_var)
    nplots = len(columns)
    ncols = min(4, nplots)
    nrows = (nplots + 3) // 4
    figsize = (2.25 * ncols, 2.25 * nrows)
    _, axes = plt.subplots(
        nrows, ncols, figsize=figsize, sharey=True, squeeze=False, layout='constrained'
    )
    axes = axes.ravel()

    for i, col in enumerate(columns):
        ax = axes[i]
        # Data to be plotted in this panel
        df_ = df_data[[col, y_var]]
        # Scatter plot with individual data points
        ax.scatter(df_[col], df_[y_var], s=8, alpha=0.3, c='steelblue')

        uniq = df_[col].nunique()
        if uniq <= 12:
            # Treat this as a categorical variable
            means = df_.groupby(col)[y_var].mean()
            xbin, ybin = means.index, means
            ax.set_xticks(sorted(df_data[col].unique()))
        else:
            # Continuous variable
            bins = pd.cut(df_[col], bins=10)
            means = df_.groupby(bins).mean()
            xbin, ybin = means[col], means[y_var]
            ax.locator_params(axis='x', nbins=4)

        # Plot bin scatter plot
        ax.plot(xbin, ybin, marker='o', c='black', lw=1, ls='--', ms=3)

        ax.set_xlabel(col)
        if i % 4 == 0:
            ax.set_ylabel(y_var)
        corr = np.corrcoef(df_data[col], df_data[y_var])[0, 1]
        ax.text(
            0.95,
            0.05,
            f'corr = {corr:.2f}',
```

```
            transform=ax.transAxes,
            va='bottom',
            ha='right',
            fontsize=8,
        )

    for ax in axes[nplots:]:
        ax.set_visible(False)
```

Part 3 — Ordinary least squares

Use the predictors listed below throughout the model comparison.

1. Implement the function `make_model(estimator)` to build a model that can be used for estimation.

   The function should build a pipeline with input standardization, polynomial expansion, a second standardization, and use the given `estimator` as the final step.

2. Check how many features a polynomial expansion with degrees 1, 2, and 3 generates.

   Hint: This number is stored in the attribute `n_output_features_` of a fitted `PolynomialFeatures` object.

3. Select the optimal polynomial degree using cross-validation with `GridSearchCV()`, specifying a parameter grid of 1, 2, and 3, `scoring='neg_root_mean_squared_error'`, and `return_train_score=True`. Use `make_model()` to build the model to be cross-validated, passing `LinearRegression()` as the estimator.

   Use a CV splitter using `KFold` with five shuffled folds with `random_state=1234`

4. Report the selected degree and CV RMSE. Use the provided `plot_ols_validation_curve()` to compare training and validation errors.

[13]:
```
def make_model(estimator):
    """Build a polynomial regression pipeline."""
    # TODO: Add input scaling, polynomial expansion, and output scaling
    # TODO: Add the supplied estimator as the final step named 'model'
    # TODO: Return the pipeline
```

[14]:
```
# Numeric predictors; living and lot area enter in logs
features = [
    'logLivingArea',
    'logLotArea',
    'YearBuilt',
    'YearRemodeled',
    'BasementArea',
    'FinishedBasementArea',
    'OverallQuality',
    'OverallCondition',
    'Bathrooms',
    'Bedrooms',
    'Fireplaces',
    'HasGarage',
]

# Training and test data for the selected features
X_train = df_train[features]
X_test = df_test[features]
```

[15]:
```
def plot_ols_validation_curve(ols_cv):
    """
    Plot training and validation RMSE against polynomial degree.
```

```
    Parameters
    ----------
    ols_cv : GridSearchCV
        Fitted GridSearchCV object for OLS regression
    """
    import matplotlib.pyplot as plt

    df_ols = pd.DataFrame(ols_cv.cv_results_)
    degrees = df_ols['param_poly__degree']
    train_score = -df_ols['mean_train_score']
    test_score = -df_ols['mean_test_score']
    _, ax = plt.subplots(figsize=(5, 3), layout='constrained')

    colors = ['#1f78b4','#e31a1c', '#33a02c']

    ax.plot(degrees, train_score, c=colors[0], marker='o', label='Training')
    ax.plot(degrees, test_score, c=colors[1], marker='o', label='Validation')
    ax.set_xticks(degrees)
    ax.set_xlabel('Polynomial degree')
    ax.set_ylabel('RMSE in log prices')
    ax.set_title('Validation curve for polynomial degree')
    ax.legend()
```

Part 4 — Ridge regression

You are now going to repeat the cross-validation task with Ridge regression.

1. Replace OLS with `Ridge()`. Select degree and the Ridge penalty alpha jointly, using degrees 1, 2, and 3 and 100 log-spaced alphas from 10^{-2} to 10^{2}. How many parameter combinations and fold fits does the search require?
2. Report the selected parameters and CV RMSE.
3. Use the provided `plot_validation_curves()` to plot the validation curves. Do different degrees favor different penalties? Check whether the selected alpha is at a grid endpoint.

[22]:
```
def plot_validation_curves(search, model_name):
    """
    Plot validation RMSE against alpha for each polynomial degree.

    Parameters
    ----------
    search : object
        Fitted grid search over polynomial degree and model alpha
    model_name : str
        Model name used in the plot title
    """
    df_results = pd.DataFrame(search.cv_results_)
    degree_col = 'param_poly__degree'
    colors = ['#1f78b4','#e31a1c', '#33a02c']

    _, ax = plt.subplots(figsize=(5.5, 3.2), layout='constrained')
    for i, (degree, df_group) in enumerate(df_results.groupby(degree_col)):
        df_group = df_group.sort_values('param_model__alpha')
        alphas = df_group['param_model__alpha']
        rmse = -df_group['mean_test_score']
        ax.plot(alphas, rmse, c=colors[i], label=f'Degree {degree}')

    best = df_results.loc[search.best_index_]
    ax.scatter(
        best['param_model__alpha'], -best['mean_test_score'], c='black', zorder=3
    )
```

```
    ax.set_xscale('log')
    ax.set_xlabel(r'Regularization strength $\alpha$')
    ax.set_ylabel('CV RMSE in log prices')
    ax.set_title(f'Validation curve for {model_name}')
    ax.legend()
```

Part 5 — Lasso

In this part, you will repeat the cross-validation task with Lasso regression.

1. Use `Lasso()` with the same predictors, pipeline transformations, folds, and scorer. Select degree jointly with 100 log-spaced alphas from 10^{-4} to 10^{-1}.
2. Report the selected parameters, CV RMSE, and number of nonzero coefficients.
3. Use the provided `plot_validation_curves()` to plot the validation curves and check the alpha grid endpoints.

Use `max_iter=1_000_000`, `tol=1e-6`, `precompute=True`, `selection='random'`, and the fixed random seed. Revisit the solver settings if convergence warnings appear.

Part 6 — Compare the results

1. Compare OLS, Ridge, and Lasso in a table showing the selected hyperparameters, CV RMSE, and test RMSE.
2. Which model would you choose using CV? Does the test ranking agree? How much difference does regularization make in this application?

Use CV for model selection and test errors to assess the fitted models. The `root_mean_squared_error()` function computes RMSE.